Algorithms for genomic data analysis

Assignment 2: metagenomics sample classification

winter semester 2025/2026

Magdalena Czapiewska

December 14, 2025

General algorithm - sketch creation

Training and testing phase

- For each **training (file, city) pair**, a **sketch** of the sample is computed.

$$\texttt{sketches_by_city[city]} = [\text{sketch}_1, \text{sketch}_2, \ldots]$$

- For each **testing FASTA file**, a **sketch** is computed.

$$\texttt{test_sketches[file]} = \text{sketch}$$

Sketch of all sequences in a file

- Every k-mer of each sequence in a file is hashed. Only the **s smallest unique hash values** are retained in the sketch.
- Parameters used: $k = 14, \quad s = 25{,}000$
- These values were chosen experimentally — increasing s or changing k did not improve the AUC-ROC of the classifier.

General algorithm - Jaccard formula

Jaccard formula

Let **A** and **B** be 2 sets of k-mers from 2 files (one training and one testing), and **S(A)** and **S(B)** be their **sketches**.

$$J(A,B) = \frac{|S(A \cup B) \cap S(A) \cap S(B)|}{|S(A \cup B)|}$$

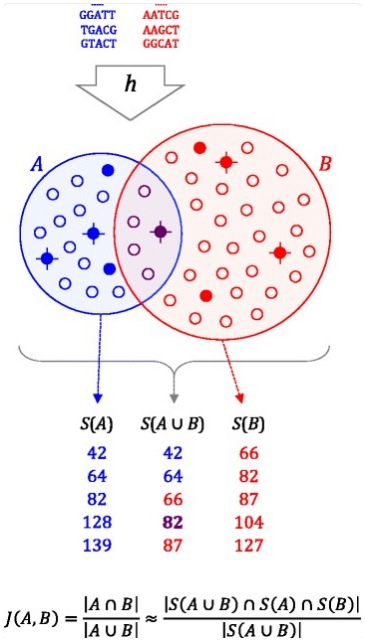

Figure: Ondov et al. 2016 "Mash: fast genome and metagenome distance estimation using MinHash"

General algorithm - similarity

Similarity computation

For every test sample:

- **Jaccard similarity** is computed between the test sketch and *every* training sketch.
- For each **city represented by N sketches**, the mean similarity is computed:

$$\text{mean Jaccard}_{\text{j, city}} = \frac{1}{N} \sum_{i=1}^{N} J(\text{test}_{\text{j}}, \text{train}_{\text{city, i}})$$

fasta_file	**City A**	**City B**	...
test1	mean Jaccard $_{1,\ \text{City A}}$	mean Jaccard $_{1,\ \text{City B}}$	...
test2	mean Jaccard $_{2,\ \text{City A}}$	mean Jaccard $_{2,\ \text{City B}}$	...
test3	...	...	...

ntHash - nucleotide hashing for k-mers

ntHash overview

- ntHash is a **hash function designed for nucleotide sequences**.
- Each k-mer in a DNA sequence is converted to a **64-bit hash value**.
- Instead of recomputing hashes from scratch, **rolling hash** allows efficient update for consecutive k-mers:

$$\text{hash}(s_{i+1} \dots s_{i+k}) \text{ from } \text{hash}(s_i \dots s_{i+k-1})$$

- Only valid nucleotides (A, C, G, T) are hashed.
- **Deterministic:** same k-mer always produces the same hash.

ntHash - calculation details

Step 1: Initial hash of the first k-mer

- Let k-mer be $s_0 s_1 \ldots s_{k-1}$.
- Each nucleotide s_i is mapped to an integer index using:

$$\mathrm{A} = 0, \mathrm{C} = 1, \mathrm{G} = 2, \mathrm{T} = 3$$

- Each index has a precomputed 64-bit **seed**.
- The **initial hash** is computed by combining rotated seeds:

$$h = \bigoplus_{i=0}^{k-1} \mathrm{rotl}(\mathrm{seed}[s_i], k-1-i) =$$

$$= \mathrm{rotl}(\mathrm{seed}[s_0], k-1) \oplus \ldots \oplus \mathrm{rotl}(\mathrm{seed}[s_{k-1}], 0)$$

where $\oplus$ is bitwise XOR and rotl is left rotation.

- **rotl(x, k)** = left rotation of bits in x by k positions:

$$\mathrm{rotl}(x, k) = (x \ll k) \mid (x \gg (64-k))$$

ntHash - calculation details

Step 2: Rolling hash for subsequent k-mers

- For the next k-mer, we **remove the effect of the outgoing nucleotide** and **add the incoming nucleotide**:

$$h_{\text{next}} = \text{rotl}(h_{\text{prev}}, 1) \oplus \text{rotl}(\text{seed}[\text{old}], k) \oplus \text{seed}[\text{new}]$$

- This allows updating the hash in **O(1) time** per k-mer, without recomputing from scratch.

Step 3: Handling invalid nucleotides

- If a k-mer contains non-ACGT characters, it is skipped.
- Hashing resumes from the next valid k-mer.

Data structures used in sketch creation

1. Set for hashes already seen in a file

- **Set ‘seen_hashes‘** stores all unique kmer hashes from a file.
- Ensures that each hash appears only once in the **sketch**.
- Operations are **O(1)**: check existence, add, or remove.

2. Heap for hashes currently in the sketch

- **List ‘sketch‘** used as a **max-heap** (‘heapq‘) to maintain the s smallest hashes.
- **Adding a new hash**:
  - If `len(sketch)` $<$ `s` – push to heap.
  - If `len(sketch)` $=$ `s` – compare with the largest hash in heap and replace if smaller (‘heapreplace‘).
- The sketch is sorted ascending at the end to easily compute the sketch of the union of 2 sets.